

Islamic University of Technology

Department of Software Engineering
Software Project Lab II (SWE 4404)

Project title

AmarShohor

(A Community-Driven Urban Issue Mapping Platform)

Final Report

Supervisor

Yousuf Ibne Kamal Niloy

Group: 02

Team Members

Procheta Silvie (230042114)
Tasnia Farzana (230042124)
Aksan Anan Ria (230042154)

Repository: https://github.com/aksanX/SPL_II_AmarShohor

Contents

1 Executive Summary 4

2 Problem Domain and Background 4

2.1 Application domain 4
2.2 The problem being solved 5
2.3 Users 5
2.4 Why it matters 5
2.5 Fundamental concepts underlying the project (CO1) 5
2.6 Specialist knowledge used 6

3 Literature and Existing Solutions 7

3.1 Existing applications 7
3.2 Technical sources 7
3.3 How AmarShohor builds on and differs from them 7

4 Requirements (SRS Session) 8

4.1 Stakeholders 8
4.2 Functional requirements and traceability 8
4.3 Non-functional requirements 10
4.4 Contextual and sustainable-development considerations 10
4.5 Feedback from the SRS session and changes made 11

5 System Design (Design Session) 11

5.1 Design approach 11
5.2 UML class diagram 12
5.3 Feature-wise three-tier architecture 13
5.4 Traceability of the design to the SRS 14
5.5 Database schema 15
5.6 Design alternatives compared 16
5.7 Feedback from the Design session and changes made 17

6 Implementation (Implementation Session) 17

6.1 Project structure 17
6.2 Implemented architecture 18
6.3 Communication between components 20
6.4 Division of classes and methods 22
6.5 Implemented features 23
6.6 Refactoring after the Implementation session 23
6.7 Fundamental concepts in the code (CO1) 24
6.8 Conformance with the design (CO3) 24

7 Tools, Version Control, Testing, and CI/CD 25

7.1 Frameworks, libraries and tools 25
7.2 Version control and workflow 26
7.3 Testing approach and results 27
7.4 CI/CD pipeline 27

8 Professional, Ethical, and Legal Considerations 28
8.1 Licences of libraries, data and assets 28
8.2 User data and privacy 29
8.3 Security measures 29
8.4 Legal and ethical issues in the deployment context 29

9 Project Management and Feasibility 30
9.1 Timeline: planned vs actual 30
9.2 Task tracking and progress review 30
9.3 Scope and schedule changes and how they were handled 34
9.4 Effort, infrastructure and third-party cost estimates 34
9.5 Feasibility 35

10 Teamwork and Individual Contributions 35
10.1 Responsibilities and evidence 35
10.2 Coordinating interdependent modules 37
10.3 Disagreements, blockers and fair participation 37

11 Results, Limitations, and Future Work 38
11.1 Implementation status of each requirement 38
11.2 Known limitations 40
11.3 Future work 40

12 Glossary and References 40
12.1 Glossary 41
12.2 References 41

1. Executive Summary

The problem. People in Dhaka and other Bangladeshi cities see broken roads, garbage piles, blocked drains, dark streetlights and open manholes every day. There is no shared, trusted place to report them. A complaint made by phone or on social media leaves no public record, the same problem is reported many times, fake or exaggerated posts are hard to tell apart from real ones, and nobody can see whether anyone is working on a fix.

Our solution. AmarShohor ("My City") is a website that works on phones and computers. A resident takes a photo of a problem and posts it. Neighbours confirm that it is real; when enough trusted people nearby agree, the problem is marked "validated" and appears on a city map that shows where problems are concentrated. Small problems, such as a garbage pile, go to volunteers, who can work alone or as a team. Large or dangerous problems, such as a broken road or a live wire, go to the City Corporation responsible for that part of the city. Whoever fixes a problem must upload an "after" photo, and residents confirm the fix before the problem is closed.

Keeping it fair and safe. The system, not a single moderator, decides most things. Votes from new accounts and from people far away count less, fake reports are hidden when enough people flag them, and every important rule is checked by the database itself, so it cannot be bypassed by editing the website. A small number of administrators handle setup and unclear cases, and every decision they make needs a written reason that is logged. For emergencies such as a fire, the app shows the national 999 number first and then lets residents warn neighbours within about 1 km.

The outcome. The team delivered a complete, working system: of the 36 functional requirements in our SRS, 31 are fully implemented and the other 5 partly; 6 user roles are supported (visitor, resident, volunteer, City Corporation official, area admin and super admin), and the code has 521 automated web tests and 4 database test scripts that run automatically on every change. The parts we did not build are listed honestly in Section 11, for example a full Bangla interface and detecting duplicate photos by image similarity.

Self-assessment

Course outcome question	Our answer and evidence
CO6: Can you communicate the project effectively to a non-technical audience without relying on unexplained jargon?	This summary uses everyday words (photo, neighbours, map, volunteers) and explains the few terms it needs ("validated" = enough trusted people agreed). Technical terms used later are defined in the Glossary (Section 12).

2. Problem Domain and Background

2.1. Application domain

AmarShohor belongs to **civic technology**: software that helps residents and local government work together on public services. In Bangladesh, cities are run by City Corporations (for example Dhaka North, DNCC, and Dhaka South, DSCC), divided into wards and thanas. Services such as roads, streetlights, drainage and waste collection are their responsibility, while other agencies (for example WASA for water and DESCO for electricity) own some infrastructure.

2.2. The problem being solved

- **No shared record:** complaints made by phone or on social media disappear; residents cannot see whether a problem was already reported or is being handled.
- **Trust:** an open reporting site attracts duplicate, exaggerated and fake posts; a single moderator cannot check them all.
- **Who should act:** small problems (litter, dumping, dengue breeding sites) can be fixed by the community, while large or dangerous ones must go to the right authority; today nobody routes them.
- **Accountability:** there is no visible measure of how long an authority takes, and no way for residents to confirm a fix is real.

2.3. Users

User	What they need
Visitor (not signed in)	See the feed, the map and issue pages to know what is happening in their area
Citizen (resident)	Report a problem quickly with a photo, support or flag others' reports, confirm fixes
Volunteer	Find nearby tasks, work alone or in a team, prove the fix, earn reputation
City Corporation official	A work queue of validated issues in their own area, with clear deadlines
Area admin	Decide unclear cases in one City Corporation area and verify its officials
Super admin	Set up City Corporations, categories and settings; supervise area admins

2.4. Why it matters

Unrepaired roads cause accidents, blocked drains cause waterlogging, and stagnant water spreads dengue. Making problems visible on a map, routing them to whoever can fix them and tracking delays publicly supports UN Sustainable Development Goal 11 (sustainable cities and communities) and Goal 16 (accountable institutions).

2.5. Fundamental concepts underlying the project (CO1)

Area	Concept	Where it is needed
Programming	Object-oriented design: abstract base class, inheritance, encapsulation, dependency injection	Backend services in the web app all extend `BaseService`
Programming	Asynchronous programming (Promises, async/await), event-driven UI (React)	Every network call, live notifications
Data structures	Hash maps (caches), queues, trees, arrays of coordinates	Place-search cache and request queue; category group → subgroup → category tree; polygon rings
Algorithms	Weighted scoring, median, exponential decay, hexagonal binning, Haversine distance, Douglas–Peucker line simplification, segment-intersection test	Validation score, severity votes, heat formula, hexagon map, distance checks, drawing City Corporation areas
Databases	Relational schema and constraints, transactions, indexes (B-tree, partial unique, GiST spatial), views, row-level security, stored procedures, triggers, scheduled jobs	All business rules and security live in PostgreSQL
Concurrency	Race conditions, atomic conditional update, row locks (`SELECT … FOR UPDATE`)	Two volunteers accepting the same task; two admins deciding the same case
Software design	Layered architecture, single responsibility, separation of concerns, publish/subscribe	Feature folders, service layer, Supabase Realtime notifications

2.6. Specialist knowledge used

- **Geographic information systems (GIS):** WGS 84 coordinates (EPSG:4326), the Web-Mercator map projection (EPSG:3857) and its stretching away from the equator, PostGIS `geography` types, point-in-polygon tests and spatial indexes. Needed because every report, City Corporation boundary and distance rule is geographic.
- **Crowdsourcing trust and reputation:** weighting votes by account age, distance and reputation to resist fake accounts (Sybil attacks), and ratio-based hiding so a small group cannot bury a real report.
- **Bangladesh urban governance:** City Corporations, wards and thanas, service hotlines, the 999 emergency number, and which agency owns which infrastructure. Needed to route each issue to the right authority.
- **Web security:** JSON Web Tokens (JWT), row-level security, least-privilege database functions, and live photo capture to stop fake evidence.
- **Mobile web media:** in-browser image compression and camera capture, because most users report from phones on slow connections.

Self-assessment

Course outcome question	Our answer and evidence
CO1: Which fundamental programming, data-structure, database and design concepts underlie the problem?	Table 2.5 lists them; Section 6.7 shows where each one appears in the code.
CO1: Which specialist knowledge is used and why is it necessary?	GIS, crowdsourcing trust, Bangladesh urban governance, web security and mobile media (Section 2.6). Without GIS the routing and map cannot work; without trust rules the community validation would be easy to abuse.

3. Literature and Existing Solutions

3.1. Existing applications

Solution	What it does	Gap that AmarShohor addresses
FixMyStreet (mySociety, UK)	Residents place a pin and a photo; the report is emailed to the responsible council	Reports are not validated by neighbours; there are no volunteers; fixes are not confirmed by the community
SeeClickFix (USA)	Residents report issues to city 311 systems; city staff update status	Depends on a city partnership; no community fixing; no weighted validation
Ushahidi	Open-source crowdmapping for crises and elections	Built for information collection, not for assigning and confirming fixes
Open311	An open standard API for submitting service requests to cities	A protocol, not an application; Bangladeshi City Corporations do not offer it
City Corporation hotlines, Facebook pages and the national 333 helpline (Bangladesh)	Phone calls or posts to the authority	No public record, no map, no duplicate detection, no way to see progress

3.2. Technical sources

- **PostGIS documentation:** `ST_DWithin`, `ST_Covers`, `geography` types and GiST indexes, used for all distance and area rules.
- **Supabase documentation:** row-level security, `SECURITY DEFINER` functions, Storage, Realtime and pg_cron.
- **Hexagonal binning (for example Uber's H3 work):** hexagons have equal distance to all neighbours, so they summarise point density more evenly than squares; our hexagon map follows this idea.

- **Douglas–Peucker algorithm:** simplifies a hand-drawn line while keeping its shape, used when an admin draws a City Corporation boundary.
- **OpenStreetMap, Photon and Nominatim usage policies:** map tiles and place search, with Nominatim limited to one request per second, which our `PlaceSearchService` enforces.

3.3. How AmarShohor builds on and differs from them

- **Community validation instead of moderator approval:** weighted votes and on-site confirmations decide whether a report is real.
- **Two fixing routes:** volunteers (alone or as a team) for small problems, the City Corporation for large or dangerous ones, chosen by category and location.
- **Evidence and confirmation:** live in-app photos for fixes and residents confirming or reopening.
- **Visible accountability:** target times per severity, an Overdue label and a public record per City Corporation.
- **Local context:** Bangla category names, Bangladesh-only locations, thana and ward boundaries, 999 shown first for emergencies.

Self-assessment

Course outcome question	Our answer and evidence
CO2: Does your SRS discuss prior solutions or credible technical sources and explain how the problem statement builds on or differs from them?	Yes: Table 3.1 compares five existing solutions, 3.2 lists the technical sources we relied on, and 3.3 states the five ways AmarShohor differs.

4. Requirements (SRS Session)

Our SRS follows the traditional format used in the course. Each requirement is a numbered "The system shall …" statement (FR-1 to FR-36), refined into user stories with a priority and acceptance criteria, and followed by a list of non-functional requirements. The full document is in `doc/srs.pdf`. This section summarises it, traces every requirement to a stakeholder need, and records how the requirements changed after the SRS session.

4.1. Stakeholders

ID	Stakeholder	Main need
S1	Citizens / residents	N1: report problems easily and see them handled
S2	Volunteers	N2: find nearby, safe tasks and get credit for real work
S3	Visitors (not signed in)	N3: see what is happening in their area without an account

ID	Stakeholder	Main need
S4	City Corporations and their officials	N4: receive the right issues for their own area, with clear deadlines
S5	Platform administrators	N5: settle unclear cases and verify officials with little effort
S6	All users	N6: trust: fake reports, fake votes and fake fixes must not win
S7	All users	N7: privacy, safety and access in Bangla and English

4.2. Functional requirements and traceability

Table 4.2 lists the 36 functional requirements exactly as named in our SRS, with the priority we gave them and the stakeholder need (Table 4.1) each one serves. Their final implementation status is in Section 11.

ID	Requirement (from the SRS)	Priority	Need
FR-1	Phone Registration: users register with their own phone number and email	High	N1, N6
FR-2	Login: users log in with their credentials	High	N1
FR-3	Profile Management: users update name, contact details and photo	Medium	N1
FR-4	Password Recovery: users recover a forgotten password	High	N1
FR-5	Volunteer Application: a citizen applies to become a volunteer	Medium	N2
FR-6	Two Language Support: interface in Bangla and English	Medium	N7
FR-7	Role Based Access: features restricted by role	High	N5, N6
FR-8	Report Submission with Attachments: photo, video or file evidence	High	N1
FR-9	Categorised Issue Reporting: each report has a category	High	N1, N4
FR-10	Duplicate Report Detection: match new reports against unresolved ones nearby	High	N6
FR-11	Report Location Management: provide and update a report's location	Medium	N1, N4
FR-12	Report Status Tracking: see the current status of a report	High	N1
FR-13	View Previous Reports: see one's own earlier reports	High	N1
FR-14	Confirm Existing Issue: citizens confirm reports that are real	Medium	N6
FR-15	Report Priority: assign priority levels to reports	High	N4, N6
FR-16	Volunteer Task Management: volunteers view and manage tasks	Low	N2
FR-17	Task Progress Update: volunteers update task progress	Medium	N1, N2
FR-18	Map-Based Report Display: reports shown by location	High	N1, N3
FR-19	Emergency Report Marking: citizens mark emergencies	High	N7

ID	Requirement (from the SRS)	Priority	Need
FR-20	Notifications: users are told about activity on reports and tasks	High	N1, N2
FR-21	Emergency Flagging and Verification: emergencies are flagged and verified	High	N6, N7
FR-22	Verify Report Evidence: evidence attached to reports is verified	High	N6
FR-23	Receive Nearby Issue Alerts: volunteers are told about nearby issues	High	N2
FR-24	Comment on a Report: comment on and share reports	Medium	N1
FR-25	Search and Filter Reports	Medium	N1, N3
FR-26	Public Analytics Dashboard: trends and summary statistics	High	N3, N4
FR-27	Escalate Unresolved Issue: critical or unresolved issues are escalated	High	N4
FR-28	Impact Info on Reported Issue: citizens describe the impact	High	N4
FR-29	Add Notes: volunteers add notes while working	High	N2
FR-30	Additional Evidence: citizens add evidence to support a report	Medium	N6
FR-31	Filter and Sort Reports	Low	N1
FR-32	Save Important Reports: save reports for later viewing	Medium	N1
FR-33	Volunteer Profile Management: skills, service area, availability	Medium	N2
FR-34	Volunteer Feedback and Rating: citizens rate completed work	Medium	N2, N6
FR-35	Task Participation Requests: volunteers request to join tasks	High	N2
FR-36	Deactivate Account	Medium	N7

4.3. Non-functional requirements

ID	Non-functional requirement (from the SRS)	Need
NFR-1	Evidence uploads are validated for size and format; unsupported files are rejected with a clear message	N1
NFR-2	The feed and map reflect new or updated reports within a few seconds without a full reload	N1, N3
NFR-3	The interface is available in Bangla and English, and the chosen language persists	N7
NFR-4	Passwords and OTP codes are stored securely; sensitive data is never exposed by a public API	N6, N7
NFR-5	Emergency reports need identity verification before broadcast, and verified emergencies are prioritised	N6, N7
NFR-6	Duplicate detection completes within a bounded search radius without delaying submission	N1, N6
NFR-7	PostgreSQL reliably persists reports, tasks, evidence, comments and user history	N1

ID	Non-functional requirement (from the SRS)	Need
NFR-8	The application stays responsive on desktop and mobile browsers, including on low bandwidth	N1, N7
NFR-9	The back end returns clear, actionable error messages on invalid input	N1
NFR-10	Role-based access control lets each role reach only the features permitted for it	N5, N6

4.4. Contextual and sustainable-development considerations

- **Low bandwidth and low-end phones:** photos are compressed in the browser before upload, and the app is a website, so nothing has to be installed.
- **Language:** category names are stored in both English and Bangla.
- **Low-participation areas:** where fewer than 15 people are active within 3 km, the validation threshold drops to 60% (minimum 2), so poorer or less connected neighbourhoods are not ignored.
- **Safety:** categories marked too dangerous for volunteers (for example open manholes and live wires) always go to an authority, and the Emergency page shows 999 first.
- **Sustainable Development Goals:** SDG 11 (sustainable cities and communities), SDG 3 (health, by reporting dengue breeding sites and waterlogging) and SDG 16 (accountable institutions, through a public City Corporation record).
- **Low running cost:** the system runs on a managed free tier during a pilot (Section 9.4).

4.5. Feedback from the SRS session and changes made

Feedback received	Change made in response
FR-15 said "authorized users" assign priority, but the SRS did not say who those users are or how to stop people inflating their own reports.	Severity is now decided by the community: it starts at the category default and, after 3 severity votes, the median wins (Table 4.2).
The SRS only had Citizen, Volunteer and Visitor roles. We were asked who handles large or dangerous problems, and who checks the people claiming to be officials.	We added City Corporation officials, one area admin per City Corporation and super admins, each with clear limits (FR-7, FR-27).
Phone registration with OTP (FR-1) needs a paid SMS gateway, which was questioned for a student project.	We used email registration and email password recovery for now and listed phone OTP as future work (Section 11).
Some non-functional requirements were vague, for example "within a few seconds" and "quickly enough".	We gave them measurable limits: 50 m duplicate radius, ±100 m GPS accuracy, 2,000 pins per view and about 0.1 s to group 5,000 issues into hexagons.

Self-assessment

Course outcome question	Our answer and evidence
CO2: Are functional and non-functional requirements distinguished and traceable to stakeholder needs?	Yes. Functional requirements (Table 4.2) and non-functional requirements (Table 4.3) are listed separately, and each row names the stakeholder need (N1–N7, Table 4.1) it serves.

5. System Design (Design Session)

In the Design session we presented an object-oriented design of AmarShohor, organised in a feature-wise three-tier architecture: **Model → Business Logic → Data Access Layer → PostgreSQL Database**. As the implementation progressed we updated the class diagram so that it matches the system we built. This section presents the updated class model and the architecture, traces them to the SRS, compares the design with alternatives and records the feedback we received.

5.1. Design approach

- **Encapsulation:** related data and behaviour are placed inside the appropriate class.
- **Inheritance and composition:** on the client, every service class inherits from an abstract `BaseService`; in the domain model, roles are attached to a `Profile` through `UserRole` instead of subclasses, and an `Assignment` is composed of its `AssignmentMember`s.
- **Single responsibility:** each class, service and repository is responsible for one part of the system.
- **Separation of concerns:** domain models, business rules, database operations and stored data are kept apart.
- **Modularity:** reporting, community validation, volunteer management, task handling and notifications are independent modules.
- **Extensibility:** new categories, notification types, rewards or analytics can be added without redesigning the system.

5.2. UML class diagram

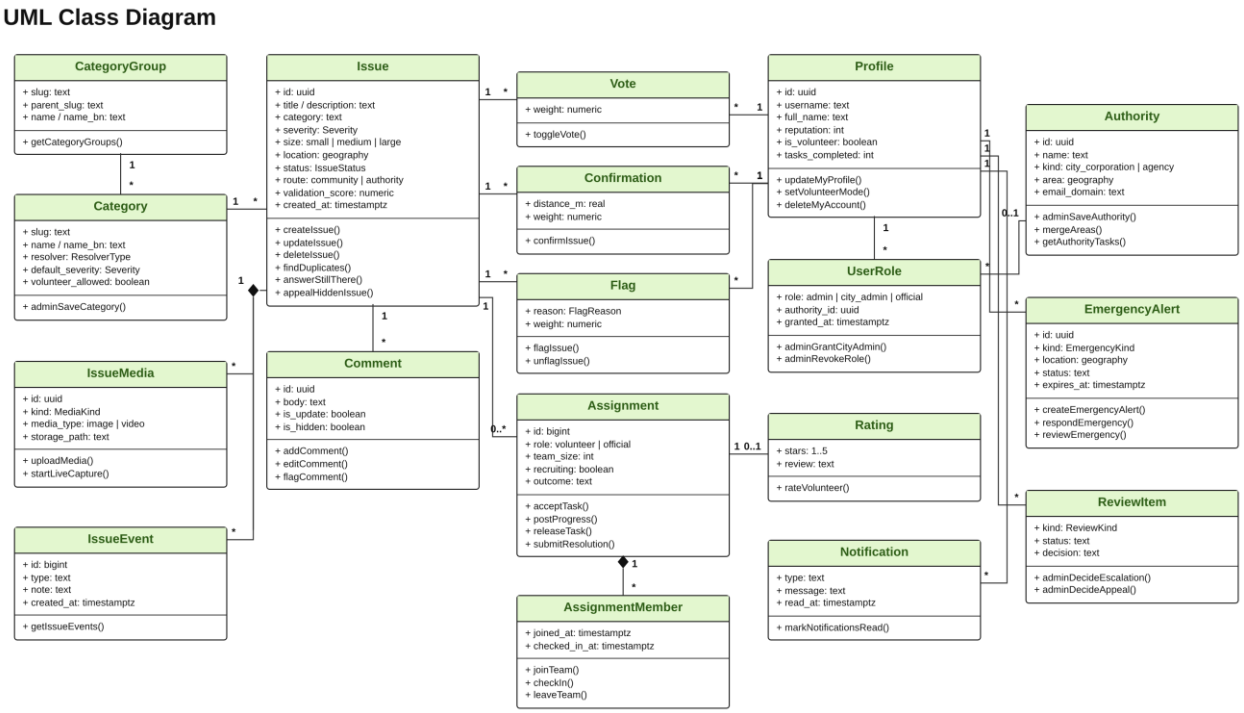


Figure 5.1: UML class diagram of AmarShohor

`Issue` is the central class. It belongs to one `Category` (grouped by `CategoryGroup`), owns its `IssueMedia` and its `IssueEvent` timeline entries, and has many `Comment`s. Community signals are separate classes that link a `Profile` to an `Issue`: `Vote`, `Confirmation` (with the distance and weight of the confirmation) and `Flag`. When someone works on an issue it gets an `Assignment` (volunteer or official), a team is made of `AssignmentMember`s, and a finished assignment can receive one `Rating`. A `Profile` can hold several `UserRole`s (admin, city_admin or official), each tied to an `Authority`: a City Corporation or agency with its own map area and official email domain. `EmergencyAlert`, `ReviewItem` (the admin review queue) and `Notification` complete the model. The operations of each class match service-class methods and database functions, for example `acceptTask()` calls `accept_task`.

Compared with the class diagram we showed in the Design session, roles are modelled with `UserRole` instead of `Citizen` and `Volunteer` subclasses (one person can be a citizen, a volunteer and an official at once), `Report` became `Issue`, `Task` became `Assignment` with `AssignmentMember` for teams, `Evidence` became `IssueMedia`, and `Feedback` became `Rating`. `Authority`, `UserRole`, `EmergencyAlert` and `ReviewItem` were added for City Corporations, admins and emergencies, following the Design-session feedback (Section 5.7).

5.3. Feature-wise three-tier architecture

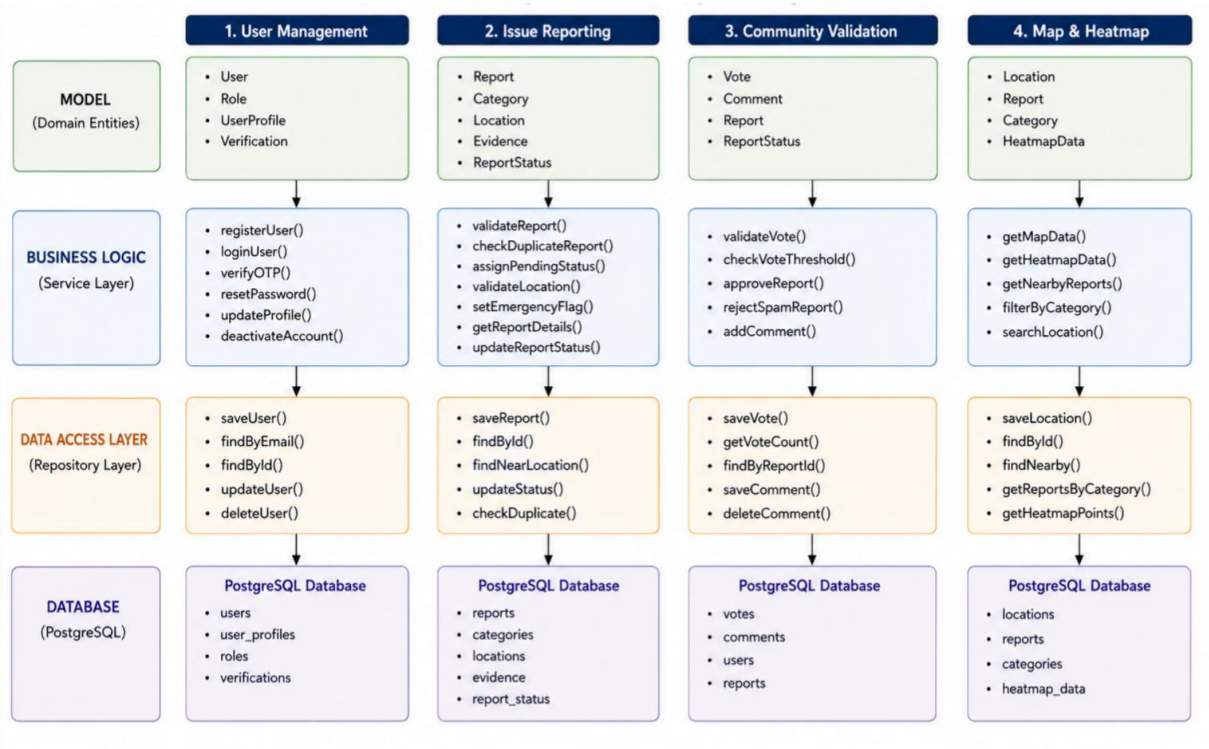


Figure 5.2: Feature-wise three-tier architecture, diagram 1 (user management, issue reporting, community validation, map and heatmap)

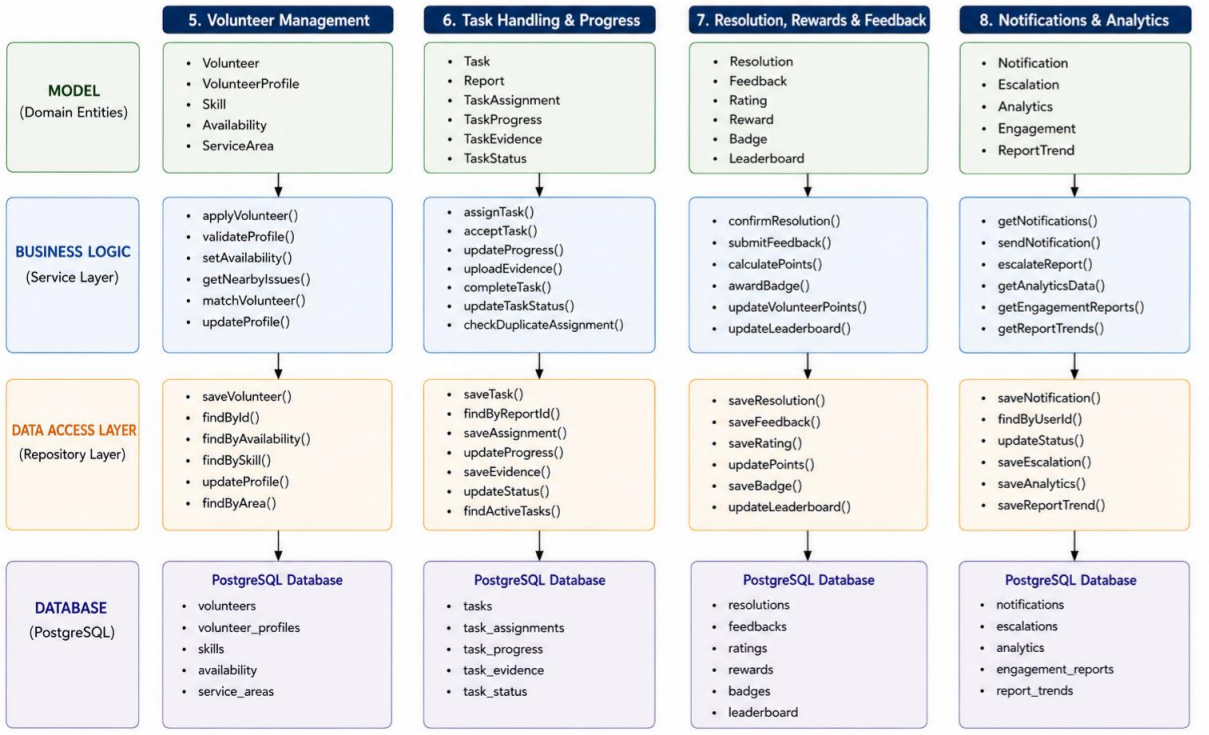


Figure 5.3: Feature-wise three-tier architecture, diagram 2 (volunteers, tasks, resolution and rewards, notifications and analytics)

In every module, the **Model** layer holds the domain entities, the **Business Logic** layer performs validation, duplicate detection, community verification, task matching, resolution and notifications, the **Data Access** layer saves and retrieves data, and **PostgreSQL** stores

everything. Keeping the layers separate means a change in one layer has little effect on the others.

5.4. Traceability of the design to the SRS

Design module (Figures 5.2, 5.3)	Main classes (Figure 5.1)	SRS requirements
User management	`Profile`, `UserRole`, `Authority`	FR-1 to FR-4, FR-7, FR-36, NFR-4, NFR-10
Issue reporting	`Issue`, `Category`, `CategoryGroup`, `IssueMedia`, `IssueEvent`	FR-8 to FR-13, FR-28, NFR-1, NFR-6
Community validation	`Vote`, `Confirmation`, `Flag`, `Comment`	FR-14, FR-15, FR-22, FR-24, FR-30
Map and heatmap	`Issue`, `Category`	FR-18, FR-25, FR-31, NFR-2
Volunteer management	`Profile`, `Notification`	FR-5, FR-23, FR-33
Task handling and progress	`Assignment`, `AssignmentMember`	FR-16, FR-17, FR-29, FR-35
Resolution, rewards and feedback	`Rating`, `Profile` (reputation)	FR-34
Notifications, escalation and analytics	`Notification`, `Authority`, `EmergencyAlert`, `ReviewItem`	FR-19 to FR-21, FR-26, FR-27, NFR-5
Saved reports	`Issue` (followed by a `Profile`)	FR-32

5.5. Database schema

The final schema has 32 PostgreSQL tables. The central table is `issues` (the `Report` class); most other tables record an action on an issue. Figure 5.4 shows the core entities.

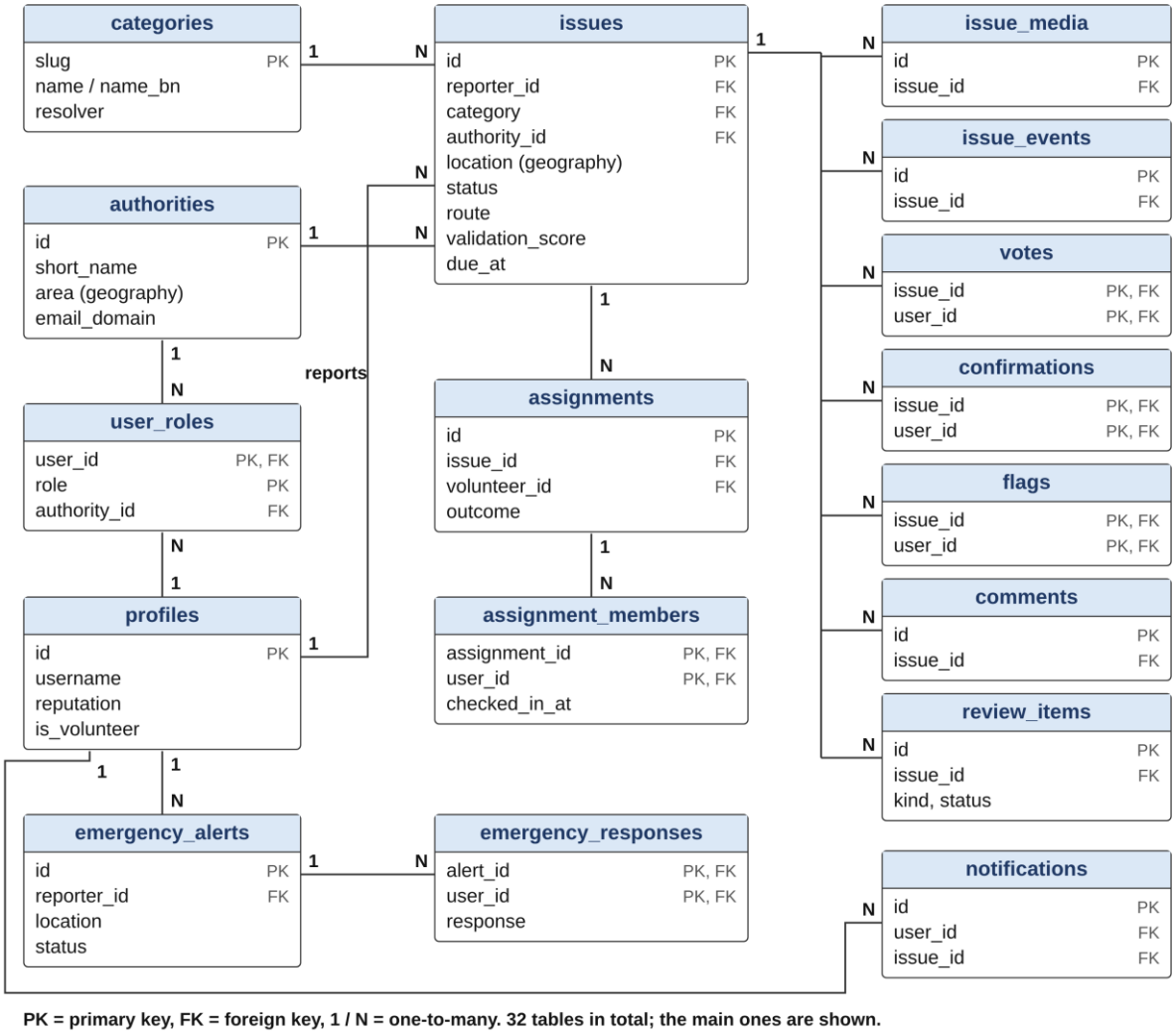


Figure 5.4: Core entities and relationships of the final database

Group	Tables	SRS requirements
Users and roles	`profiles`, `user_settings`, `user_roles`, `role_requests`	FR-1 to FR-7, FR-33, FR-36
Issues	`issues`, `issue_media`, `issue_events`, `categories`, `category_groups`, `category_suggestions`	FR-8 to FR-13, FR-28
Community signals	`votes`, `confirmations`, `severity_votes`, `flags`, `dismissed_flags`, `follows`, `still_there_answers`, `comments`, `comment_flags`	FR-14, FR-15, FR-22, FR-24, FR-30, FR-32
Fixing	`assignments`, `assignment_members`, `resolution_reviews`, `ratings`, `authorities`	FR-16, FR-17, FR-27, FR-29, FR-34, FR-35
Emergencies	`emergency_alerts`, `emergency_responses`, `alert_updates`	FR-19, FR-21, NFR-5
Admin and system	`review_items`, `admin_actions`, `notifications`, `capture_tokens`, `app_settings`	FR-20, FR-23, FR-26

Key decisions: counters such as `upvote_count` and `validation_score` are stored on the issue row so the feed reads one row; locations use the PostGIS `geography` type with spatial (GiST) indexes; a partial unique index allows only one active assignment per issue; and `app_settings` is a single-row table holding every threshold.

5.6. Design alternatives compared

Criterion	A. Node/Express API with a repository layer + PostgreSQL (Design-session plan)	B. Supabase with business rules in PostgreSQL functions (chosen)	C. Firebase (Firestore NoSQL)
Trust (need N6)	Rules in the API server; safe only if every route checks them	Rules run in the same transaction as the data change and cannot be bypassed	Security rules are limited; counters are hard to keep consistent
Geographic queries	PostGIS available	PostGIS available	No native polygon containment
Usability (live updates)	Needs our own WebSocket server	Realtime built in	Realtime built in
Scalability	We must scale the API server	Managed database and API; static front end	Scales well, but cost per read grows fast
Resource use and team effort	Extra code for auth, uploads and sockets	Auth, storage, realtime and cron provided	Provided, but the data model fights relational rules
Lifecycle cost	A server to host and patch	Free tier for a pilot; about USD 25 per month later	Pay per read and write; hard to predict

Decision. We chose B. Trust (N6) is the most important need, and rules inside the database make every action atomic and impossible to bypass from a modified browser. PostGIS was required to route issues by City Corporation area, which rules out C. B also removed the separate server for login, uploads and live updates, which a team of three could not have built and tested in the time available. The layers of the design are kept: the Model became TypeScript interfaces in `models/`, the Data Access layer became service classes, and the Business Logic moved into database functions (Section 6.2).

We made a second, smaller choice for the map: **Leaflet with OpenStreetMap** (free, open licence, no billing key) instead of the Google Maps JavaScript API, which has better geocoding but needs a billing account.

5.7. Feedback from the Design session and changes made

Feedback received	Change made in response
We were asked whether a separate API server and repository layer were needed, since the database could check the same rules itself.	We moved the business logic into PostgreSQL functions on Supabase and compared the options explicitly (Table 5.6).

Feedback received	Change made in response
The design had no one to verify officials or settle disputed cases, and a single admin could not cover every City Corporation.	We added City Corporations, officials, one area admin per City Corporation and super admins (migrations 0007–0010 and 0044–0052).
Reviewers asked how the domain classes would map to real code and tables.	Section 6.8 maps every design class to its table and service class; the client was built around an abstract `BaseService`.

Self-assessment

Course outcome question	Our answer and evidence
CO3: Does your design include a justified architecture and component/data models that trace back to the SRS?	Yes. Figures 5.1 to 5.4 give the class model, the architecture and the database; Tables 5.4 and 5.5 trace them to FR and NFR numbers.
CO3: Did you compare plausible alternatives and justify the selected design?	Yes. Table 5.6 compares three architectures on trust, usability, scalability, resource use and lifecycle cost, followed by the decision and its reasons.

6. Implementation (Implementation Session)

The final software has 210 TypeScript source files (about 14,100 lines, excluding tests) and 78 ordered SQL migration files (about 15,300 lines), with 67 web test files and 4 SQL scenario test scripts.

6.1. Project structure

```
SPL_II_AmarShohor/
├── README.md                  how to build, run and test
├── .github/workflows/ci.yml   automatic tests on every push
├── doc/                       srs.pdf, design.pdf, final-report.pdf, guides/
└── code/
    ├── supabase/
    │   ├── migrations/        78 SQL files, run in order (0001 → 0078)
    │   ├── seed.sql           development data (City Corporation areas)
    │   └── tests/             run_local.sh + scenario.sql … scenario_v4.sql
    └── web/
        ├── public/data/       city-thanas.json (thana outlines)
        └── src/
            ├── main.tsx       entry point
            ├── app/           routes, providers, layout (top bar, sidebars)
            ├── core/          supabase.ts, api/AppError.ts, api/BaseService.ts
            ├── models/        14 files of data types (issue, map, role, …)
            ├── shared/        ui/, hooks/, lib/, geo/, settings/
            ├── test/          shared test helpers
            └── features/      17 feature folders:
                admin, auth, categories, city-corp, comments, emergency,
                feed, issues, landing, leaderboard, legal, map, media,
                notifications, profile, settings, volunteer
                (each: pages/ components/ hooks/ services/ lib/)
```

Why this organisation. The code is split **by feature**, so everything for one part of the app (for example the map) is in one folder. Inside a feature, files are grouped by kind: `pages/` (routed

screens), `components/` (UI), `hooks/` (React state), `services/` (backend calls) and `lib/` (plain, unit-tested logic). Dependencies point one way: `app` → `features` → `shared` → `core`, and `core` and `shared` never import from a feature. Database changes are append-only numbered migrations, so every developer and the CI rebuild the same database from scratch.

6.2. Implemented architecture

The implementation keeps the layers of the design but places them differently. The React application is the presentation layer; the service classes are the data-access layer; the business logic runs as PostgreSQL functions inside Supabase; and PostgreSQL with PostGIS stores the data. There is no separate application server: the browser calls the database functions through the Supabase client, and each function checks the caller and the rules before changing anything.

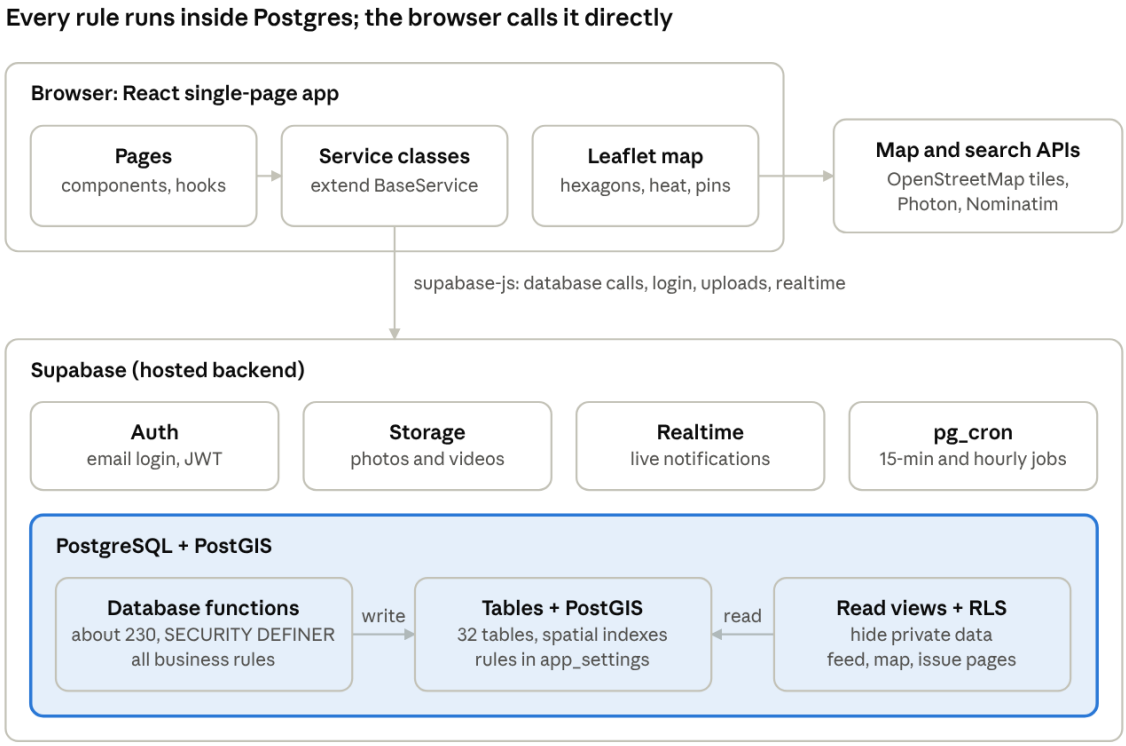


Figure 6.1: Implemented architecture and components

Component	Responsibility	SRS requirements
React pages and components	Screens, forms, map layers, photo compression, client-side checks	All FRs; NFR-8
Service classes (`BaseService` subclasses)	One named method per backend call; error translation	NFR-9
Supabase Auth	Email and password login, JWT sessions, password recovery	FR-1, FR-2, FR-4, NFR-4
Supabase Storage	Photos and videos with random file names	FR-8, FR-22, FR-30, NFR-1
Supabase Realtime	Pushes new notifications to open browsers	FR-20, FR-23, NFR-2

Component	Responsibility	SRS requirements
PostgreSQL functions (SECURITY DEFINER)	Validation, routing, locks, reputation, moderation	FR-10, FR-14 to FR-17, FR-27, FR-34, FR-35, NFR-10
Read views and row-level security	Public reads without private columns	FR-12, FR-13, NFR-4
pg_cron jobs	Lock expiry, auto-close, "still there?", passing late cases up, cleanup	FR-16, FR-27
OpenStreetMap, Photon, Nominatim	Map tiles and place search	FR-18, FR-25

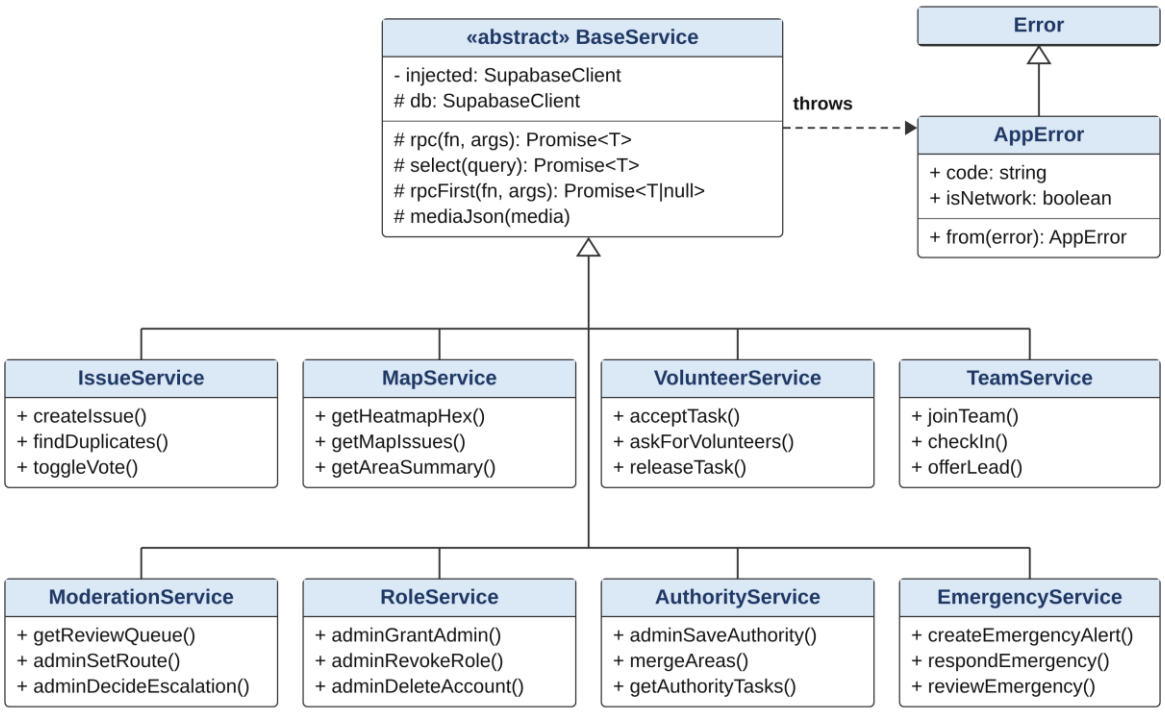


Figure 6.2: Service class hierarchy in the web application (8 of the 17 services shown, 3 methods each)

Class	Kind	Purpose
`BaseService`	Abstract class	Owns the Supabase client; protected `rpc`, `select`, `rpcFirst` and `mediaJson` turn database errors into `AppError`
17 services, e.g. `IssueService`, `MapService`, `VolunteerService`, `TeamService`, `ModerationService`, `RoleService`, `AuthorityService`, `EmergencyService`	Subclasses of `BaseService`	One service per feature; each public method is one named backend call

Class	Kind	Purpose
`AppError`	Subclass of `Error`	Carries the SQL hint as `code` and an `isNetwork` flag
`LocationService`, `PlaceSearchService`	Classes	Keep the last GPS fix; own the place-search request queue and caches
`models/*.ts` (14 files)	TypeScript interfaces	The data classes the database returns (e.g. `Issue`, `TeamMember`)

6.3. Communication between components

From → To	Channel	Message format
React component → service class	TypeScript method call	Typed arguments; returns a typed model from `models/`
Service class → Supabase	supabase-js over HTTPS (PostgREST `/rest/v1/rpc/<function>`)	JSON body of named parameters (`{"p_issue": "…"}`) + the user's JWT
PostgREST → PostgreSQL function	SQL function call as the logged-in user	Typed SQL parameters; `auth.uid()` taken from the JWT
PostgreSQL function → tables	SQL inside one transaction	Rows; errors raised with a machine code in `HINT` (e.g. `TASK_TAKEN`)
Supabase → browser (reads)	HTTPS, views and `get_*` functions	JSON arrays matching the `models/` interfaces
Browser → Storage	HTTPS upload	File bytes at `u/<random>.<ext>`; path then passed to a function as `{path, type, token}`
Database → browser (live)	Supabase Realtime (WebSocket)	New `notifications` row as JSON
Browser → OpenStreetMap / Photon / Nominatim	HTTPS	PNG map tiles; GeoJSON / JSON search results

Data classes exchanged (example)

The front end and back end share one vocabulary: each SQL view or function returns rows whose columns match a TypeScript interface in `models/`. For example, an issue returned by `get_issue` is typed as:

```
// code/web/src/models/issue.ts (excerpt)
export interface Issue {
  id: string
  title: string
  category: string | null
  resolver: 'community' | 'authority' | null
  severity: Severity            // 'low' | 'medium' | 'high' | 'critical'
  lat: number
  lng: number
  status: IssueStatus           // 'community_review' | 'validated' | 'escalated' | …
  is_anonymous: boolean
```

```
  reporter_username: string | null   // null when the report is anonymous
  upvote_count: number
  confirmation_count: number
  …
}
```

End-to-end example: a volunteer accepts a task

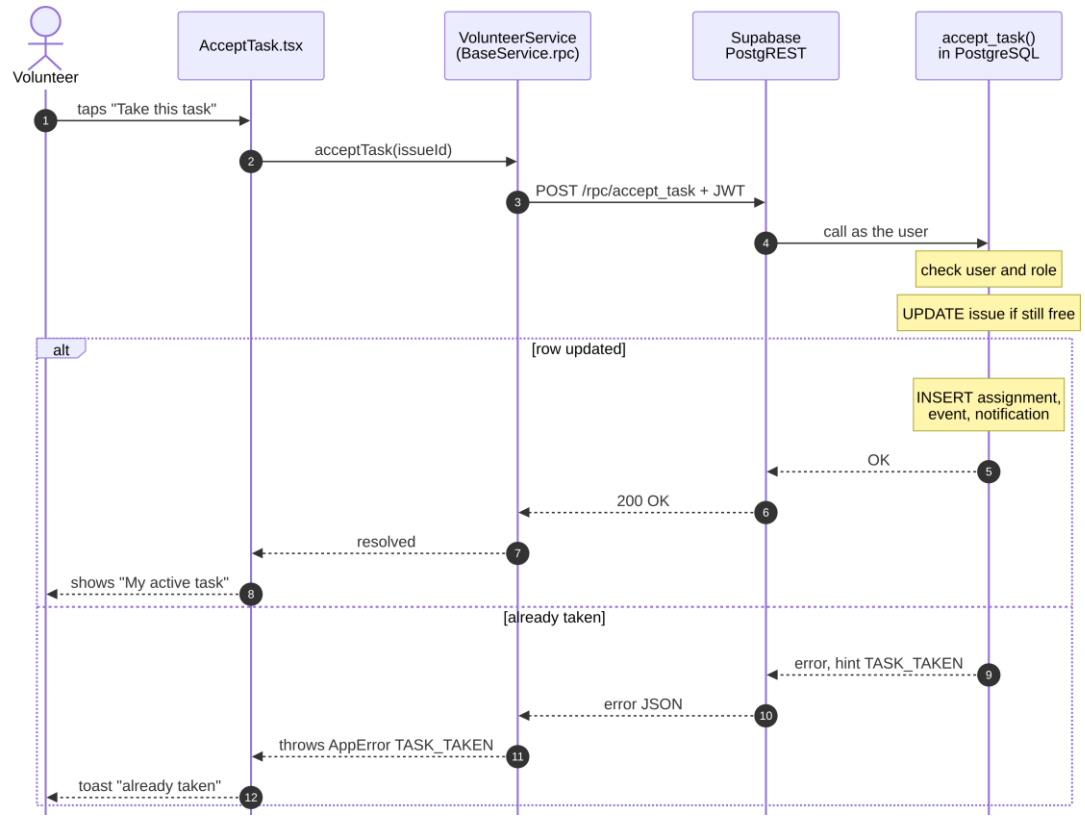


Figure 6.3: Sequence of "Take this task", including the race-condition branch

```
// 1. UI: features/volunteer/components/task-panel/AcceptTask.tsx
await volunteerService.acceptTask(issue.id)

// 2. Service: features/volunteer/services/volunteer.service.ts
acceptTask = (id: string, teamSize = 1) =>
  this.rpc<void>('accept_task', { p_issue: id, p_team_size: teamSize })

// 3. Base class: core/api/BaseService.ts
protected async rpc<T>(fn: string, args = {}): Promise<T> {
  const { data, error } = await this.db.rpc(fn, args)
  if (error) throw AppError.from(error)      // hint → error.code
  return data as T
}

-- 4. Database: accept_task(), migration 0009 (simplified)
update issues
   set volunteer_id = v_user, status = 'assigned', assigned_at = now(),
       lock_expires_at = now() + make_interval(hours => s.lock_hours)
 where id = p_issue and status = 'validated' and volunteer_id is null;
get diagnostics v_rows = row_count;
if v_rows = 0 then
  raise exception 'This task is no longer available' using hint = 'TASK_TAKEN';
end if;
```

If two volunteers tap at the same moment, only one `UPDATE` matches the `volunteer_id is null` condition. The other receives the `TASK_TAKEN` hint, which `AppError` exposes as

`error.code`, and the UI shows a message instead of a broken state. A partial unique index on active assignments is a second guard.

6.4. Division of classes and methods

Rationale.

- **Layering:** UI components never talk to Supabase directly; they call a service. Business rules live only in SQL functions. This keeps each layer replaceable and testable.
- **Single responsibility:** one service class per feature (17 services); one SQL function per user action (e.g. `accept_task`, `release_task`, `admin_decide_escalation`); small helpers for repeated checks (`require_admin`, `require_reason`, `find_authority`).
- **Reuse through inheritance:** error handling is written once in `BaseService`; every service inherits it.
- **Testability:** services take an optional client in the constructor (dependency injection), and calculations live in `lib/` as pure functions with tests next to them.
- **React guidance:** components and hooks stay functions; classes are used only where they hold state or share behaviour (services, `AppError`, `LocationService`, `PlaceSearchService`).

Lines of code per class and per method

Measured on the final code with the TypeScript compiler API (from the first to the last line of each declaration, test files excluded) and, for SQL, on the latest definition of each database function across all migrations.

Unit	Count	Average LOC	Median LOC	Maximum LOC (where)
TypeScript classes	22	41.0	26	244 (`PlaceSearchService`)
Class methods	146	4.4	2	35 (`PlaceSearchService.searchPhoton`)
All TypeScript functions (incl. React components and hooks)	566	20.4	7	397 (`MapPage` component)
SQL database functions	217	23.2	16	131 (`run_maintenance`)

Justification of the largest units. Most units are small: 78% of TypeScript functions have 20 lines or fewer, and 74% of SQL functions have 30 lines or fewer. `PlaceSearchService` is the largest class because it owns two search providers, a rate-limited queue and two caches that must share private state. `MapPage` is long because it is mostly JSX that wires the map, its layers, filters and panels together; the calculations it uses are already split into `map/lib/` (hex grid, heat, viewport, URL filters). `run_maintenance` is one scheduled job that must run its sweeps (reminders, expired locks, auto-close, expiry) in a fixed order in one transaction. Splitting the

remaining large React components (`MapPage`, `ReportForm`, `IssueCard`) further is listed as future refactoring.

6.5. Implemented features

Feature area	Implemented
Accounts	Sign-up, login, password reset, CAPTCHA, profile, settings, delete account, Terms and Privacy pages
Reporting	Photos/videos with compression, GPS accuracy check, manual pin, duplicate check, drafts, anonymity, daily limit
Community validation	Weighted upvotes, live on-site confirmations, severity votes, flags with fair-flagging, category corrections, appeals, comments
Fixing	Routing by category and location, volunteer tasks with locks, teams with check-in and leader hand-over, live "after" photos, community confirmation, reopen limits, ratings, reputation, leaderboard
City Corporations	Area drawing from thanas, wards, freehand or corners; official sign-up by email domain; official dashboard; target times and Overdue; public record; agencies
Admins	Super admins and one area admin per area; review queue; passing late cases up; activity log; categories and settings editors; unused-upload cleanup
Map	Hexagons, heat and clustered pins; category and time filters; place search; area summary; shareable URL; dark mode
Emergencies	999 and local numbers first; live-evidence alerts; neighbour confirm/deny; official updates; admin removal of fake alerts; issue-to-alert conversion
Notifications	Live in-app notifications; weekly cleanup

6.6. Refactoring after the Implementation session

Refactoring	Why
Web code reorganised from file-kind folders (`pages/`, `components/`, `lib/`) into 17 feature folders (11 Oct 2026, documented in `doc/guides/RESTRUCTURE_PLAN.md`)	To follow one feature you had to open five folders; one feature now lives in one folder
`AdminPage.tsx` (1,102 lines) split into a shell plus tab files; `VolunteerPanel.tsx` (762 lines) split into ~14 task-panel components	Single responsibility; files small enough to review
`lib/api.ts` (471 lines, ~150 calls) turned into 17 service classes extending `BaseService`	Encapsulation, shared error handling, dependency injection for tests
`lib/types.ts` (659 lines) split into 14 files in `models/`; `lib/geo.ts` split into `LocationService` and `PlaceSearchService`	One file per domain; private state kept inside classes
Repository moved into `code/` and `doc/` (27 Oct 2026)	To follow the course's required repository layout

Refactoring	Why
Migrations renumbered after merges; functions closed by default (0041); edge-case fixes (0074–0078)	Keep one ordered history for all three members; least privilege

6.7. Fundamental concepts in the code (CO1)

Concept (Section 2.5)	Where it appears in the code
Inheritance, encapsulation, dependency injection	`core/api/BaseService.ts` and its 17 subclasses; `protected rpc()`; `constructor(client?)`
Hash map cache and queue	`shared/geo/placeSearch.service.ts`: results cached per text and per ~5 km area; Nominatim queue at 1 request/second
Tree	`features/categories/lib/categoryTree.ts`: group → subgroup → category
Haversine distance	`shared/geo/distance.ts` (Earth radius 6,371,000 m)
Douglas–Peucker and segment intersection	`features/map/lib/outline.ts`: `simplifyPath()`, `ringCrossesItself()`
Hexagonal binning	`features/map/lib/hexGrid.ts` and SQL `heatmap_hex`
Exponential decay, weighted sum	SQL `issue_heat_weight`: severity × (1 + ln(1 + confirmations) + 0.25·ln(1 + upvotes)) × max(0.35, 0.5^(age/30 days))
Spatial queries and GiST index	`find_authority`: `ST_Covers(area, point)` ordered by `ST_Area`; `ST_DWithin` for 50 m / 200 m / 3 km rules
Atomic conditional update (race condition)	`accept_task` (Section 6.3)
Row locks	Migration 0033: each admin decision locks its review item with `FOR UPDATE`
Partial unique index	`review_items_one_open (issue_id, kind) WHERE status = 'open'`; one open case per issue and kind
Publish/subscribe	Insert into `notifications` → Supabase Realtime → `useNotifications` hook

6.8. Conformance with the design (CO3)

Table 6.8 maps each class of the class diagram (Figure 5.1) to its database table and the service class that uses it.

Class (Figure 5.1)	Database table(s)	Web service class
`Profile`	`profiles`, `user_settings`	`ProfileService`, `AccountService`
`UserRole`	`user_roles`	`RoleService`
`Authority`	`authorities`	`AuthorityService`
`Category`, `CategoryGroup`	`categories`, `category_groups`	`CategoryService`

Class (Figure 5.1)	Database table(s)	Web service class
`Issue`	`issues`	`IssueService`, `FeedService`, `MapService`
`IssueMedia`	`issue_media`, `capture_tokens`	`MediaService`
`IssueEvent`	`issue_events`	`IssueService`
`Comment`	`comments`, `comment_flags`	`CommentService`
`Vote`, `Confirmation`, `Flag`	`votes`, `confirmations`, `flags`	`IssueService`
`Assignment`, `AssignmentMember`	`assignments`, `assignment_members`	`VolunteerService`, `TeamService`
`Rating`	`ratings`	`VolunteerService`
`Notification`	`notifications`	`NotificationService`
`EmergencyAlert`	`emergency_alerts`, `emergency_responses`	`EmergencyService`
`ReviewItem`	`review_items`	`ModerationService`

- **Follows the design:** every class in Figure 5.1 maps one-to-one to a database table and a service class, and the code keeps the layers of the architecture (Model, Business Logic, Data Access, PostgreSQL).
- **Justified departures:** (1) the business logic runs in PostgreSQL functions instead of a separate server, for the reasons in Section 5.6; (2) the class diagram changed after the Design session (Section 5.2) to add City Corporations, officials and admins and to replace user subclasses with roles; (3) fix and "I see this too" photos became live in-app captures, because gallery photos and browser GPS can be faked.

Self-assessment

Course outcome question	Our answer and evidence
CO1: Show how the fundamental concepts named in Section 2 appear in the actual code.	Table 6.7 maps each concept to a file or SQL function.
CO3: Show that the implemented structure follows, or justifiably departs from, the design.	Table 6.8 maps every design class to its table and code and states where and why it departs.

7. Tools, Version Control, Testing, and CI/CD

7.1. Frameworks, libraries and tools

Tool	Why we chose it	Alternative considered	Limitation
React 19 + TypeScript	Component model, large ecosystem; types catch errors before running	Vue, Angular; plain JavaScript	Large components can grow long (Section 6.4)
Vite 8	Very fast dev server and build	Create React App (deprecated), webpack	Configuration differs from older tutorials
Tailwind CSS 4	Consistent styling without separate CSS files; dark mode	Bootstrap, CSS modules	Long class lists in JSX
Supabase (PostgreSQL, PostGIS, Auth, Storage, Realtime, pg_cron)	Relational rules + spatial queries + auth in one managed service	Own Node/Express server; Firebase	Business logic in SQL is harder to debug; vendor features
Leaflet + react-leaflet, leaflet.heat, leaflet.markercluster	Free, open, light; works with OpenStreetMap	Google Maps JS API, Mapbox GL	Raster tiles; very large datasets need server aggregation (done in PostGIS)
TanStack Query 5	Caching, refetching and loading states for server data	Redux Toolkit Query, plain useEffect	Another concept for new members to learn
Vitest 5 + React Testing Library + jsdom	Same configuration as Vite; tests screens like a user	Jest	jsdom is not a real browser (no camera or GPS)
oxlint	Very fast linter	ESLint	Fewer plugins than ESLint
Git, GitHub, GitHub Actions	Pull requests, history, free CI	GitLab CI	CI minutes are limited on free accounts

7.2. Version control and workflow

- The code is in one GitHub repository with **683 commits** (4 Oct – 28 Oct 2026) and **57 pull requests** (#1 to #57, all closed).
- Branches: Procheta worked on branch `p` (merged through pull requests), Tasnia on branches `Nuzhat` and `Nuzhat2.0` (17 pull requests), and Aksan committed directly to `main` for database and documentation work. Every merge to `main` runs CI.
- Commit messages describe the change in plain words (for example "Database: fix edge cases in teams, roles, removed accounts and switched-off City Corporations (0075, 0076)").
- Database changes are never edited after they run; each change is a new numbered migration, renumbered when two branches added migrations at the same time.

The version-control history can be checked directly on GitHub using the links in Table 7.2.

Evidence	What it shows	Link
Pull requests	All 57 closed pull requests, with who opened and merged each one	https://github.com/aksanX/SPL_II_AmarShohor/pulls?q=is%3Apr+is%3Aclosed
Commit history	Every commit on the main branch, with its author and date	https://github.com/aksanX/SPL_II_AmarShohor/commits/main
CI runs	The result of the automatic tests for every push and pull request	https://github.com/aksanX/SPL_II_AmarShohor/actions

7.3. Testing approach and results

Level	Tool	What it covers	Result
Unit	Vitest	Hex grid, heat, viewport, URL filters, outline, distance, formatting, categories, profile, auth, volunteer logic	67 test files, 521 tests: all passed (run on 1 Nov 2026, 125.7 s)
Component	React Testing Library + jsdom	Map page and panels, profile, settings, volunteer, leaderboard, notifications, City Corporation, login, comments, media picker, live camera, error pages	Included in the 521 tests above
Service	Vitest with a fake Supabase client	Each service calls the right database function with the right arguments	Included above
Integration (database)	SQL scenarios on PostgreSQL 15 + PostGIS 3.4	`scenario.sql` (v1 flow), `scenario_v2.sql` (roles, routing, teams, emergencies),	Run by CI on every push; ends with "All checks passed"

Level	Tool	What it covers	Result
		`scenario_v3.sql` (live evidence, flag limit, delete account, permissions), `scenario_v4.sql` (area admins)	
Manual	`doc/guides/END_TO_END_TEST.md`	One issue from report to fixed with 4 real accounts, the City Corporation route, a fake report with an appeal, and the map	Performed before the demonstration

7.4. CI/CD pipeline

`.github/workflows/ci.yml` runs two jobs in parallel on every push to `main` and every pull request:

1. **web:** `npm ci` → lint (`oxlint`) → unit and screen tests (`vitest run`) → type check and production build (`npm run build`).
2. **database:** starts a fresh PostgreSQL + PostGIS container, applies all 78 migrations in order and runs the four SQL scenarios.

Deployment (CD), planned: the production build is a static site, deployable to a static host connected to the repository, with the hosted Supabase project as the back end; `doc/guides/LAUNCH_CHECKLIST.md` lists the settings to change before real users join (email confirmation, CAPTCHA, password length, keys) and `BACKUPS.md` the weekly database backups.

Self-assessment

Course outcome question	Our answer and evidence
CO4: Can you justify the selected frameworks, libraries and tools against at least one plausible alternative and explain their limitations?	Yes: Table 7.1 gives the reason, an alternative and a limitation for each tool.
CO4: Is the codebase under version control with a meaningful history, and are testing and CI/CD practices in place?	Yes: 683 commits and 57 pull requests (7.2); 521 passing web tests plus 4 SQL scenario scripts (7.3); a two-job CI pipeline on every push (7.4).

8. Professional, Ethical, and Legal Considerations

8.1. Licences of libraries, data and assets

Component	Licence	What it requires of us
React, React DOM, React Router, Vite, Tailwind CSS, TanStack Query, supabase-js, leaflet.markercluster, date-fns, clsx, browser-image-compression, Vitest, oxlint, Testing Library, jsdom	MIT	Keep the copyright notice; free for any use
Leaflet, leaflet.heat	BSD-2-Clause	Keep the copyright notice
lucide-react (icons)	ISC	Keep the notice
TypeScript	Apache-2.0	Keep the notice; patent grant
react-leaflet	Hippocratic License 2.1	An ethical-source licence: the software must not be used to violate human rights. Our use (public civic reporting) complies; any future commercial or government deployment must review it
PostgreSQL / PostGIS	PostgreSQL License / GPL-2.0	Used as a server; we do not distribute modified PostGIS
OpenStreetMap map data and tiles	ODbL 1.0; tile usage policy	Show "© OpenStreetMap contributors"; heavy traffic needs our own or a paid tile provider
Nominatim / Photon search	Usage policies	At most 1 Nominatim request per second (enforced in `PlaceSearchService`); identify the application
Thana outlines (geoBoundaries / BBS / OCHA ROAP)	CC BY 3.0 IGO	Attribution, given in the README

8.2. User data and privacy

- **Data minimisation:** we store email, username, optional name, picture, bio and home area; location is stored only for reports, confirmations and the optional home area.
- **Anonymous reports are truly anonymous:** read views remove the reporter, and uploads are named `u/<random>.<ext>` so a photo link cannot reveal who posted it.
- **Home locations are never shown** to other users; they are used only inside database functions to find nearby volunteers.
- **Right to leave:** "Delete my account" removes personal details and closes the login; reports stay without a name.
- **Retention:** read notifications are deleted after 90 days and unread ones after a year; unused uploads can be cleared.
- **Transparency:** Terms of Use (`/terms`) and Privacy Policy (`/privacy`) pages are linked from sign-up, the landing page and Settings.

8.3. Security measures

- No table accepts writes from the browser; every write goes through a `SECURITY DEFINER` function with a fixed `search_path` that checks the caller (`auth.uid()` from the verified JWT) and their role.
- Functions are closed by default (migration 0041); a test fails if any function is open to everyone.
- Passwords are hashed by Supabase Auth; CAPTCHA on sign-up; rate limits (10 reports and 30 flags a day; posting pause for repeated fake reports).
- Live evidence: one-time server codes stamped on photos, which must be uploaded within 2 minutes and near the issue.
- Row locks stop two admins from deciding the same case at once; every admin action is logged with a reason.

8.4. Legal and ethical issues in the deployment context

- **Not an emergency service:** the app shows 999 first and states that it does not replace emergency services.
- **Defamation and people in photos:** reports are about public places, not people; comments and reports can be flagged and hidden; admins can remove content with a logged reason.
- **Fairness:** there is deliberately no downvote, so people cannot bury real problems they dislike; low-participation areas get a lower threshold; honest category mistakes cost no reputation.
- **Accountability of power:** admins cannot assign tasks, mark issues fixed or change votes or reputation; area admins cannot also be officials, so nobody judges complaints about their own work.
- **Official verification:** officials must sign up with their City Corporation's email domain and be approved, so nobody can impersonate an authority.
- **Data-protection law:** before a public launch the privacy policy must be reviewed against the data-protection law in force in Bangladesh at that time.

Self-assessment

Course outcome question	Our answer and evidence
CO4: Have you checked relevant licence, privacy, security, legal and professional-ethics requirements for the tools, data and deployment context?	Yes: Table 8.1 lists every licence and what it requires (including the Hippocratic licence of react-leaflet and the OpenStreetMap ODbL); 8.2–8.4 cover privacy, security and the legal and ethical issues.

9. Project Management and Feasibility

9.1. Timeline: planned vs actual

Table 9.1 shows the SPL-II sessions. Each session was held on its planned date, so the project kept to the course schedule from the SRS to the final submission.

SPL-II milestone	Planned	Actual
SRS session	8 Jul 2026	8 Jul 2026
Design session	22 Jul 2026	22 Jul 2026
Implementation session	12 Aug 2026	12 Aug 2026
Final presentation and demonstration	28 Oct 2026	28 Oct 2026
Final report and repository submission	1 Nov 2026	1 Nov 2026

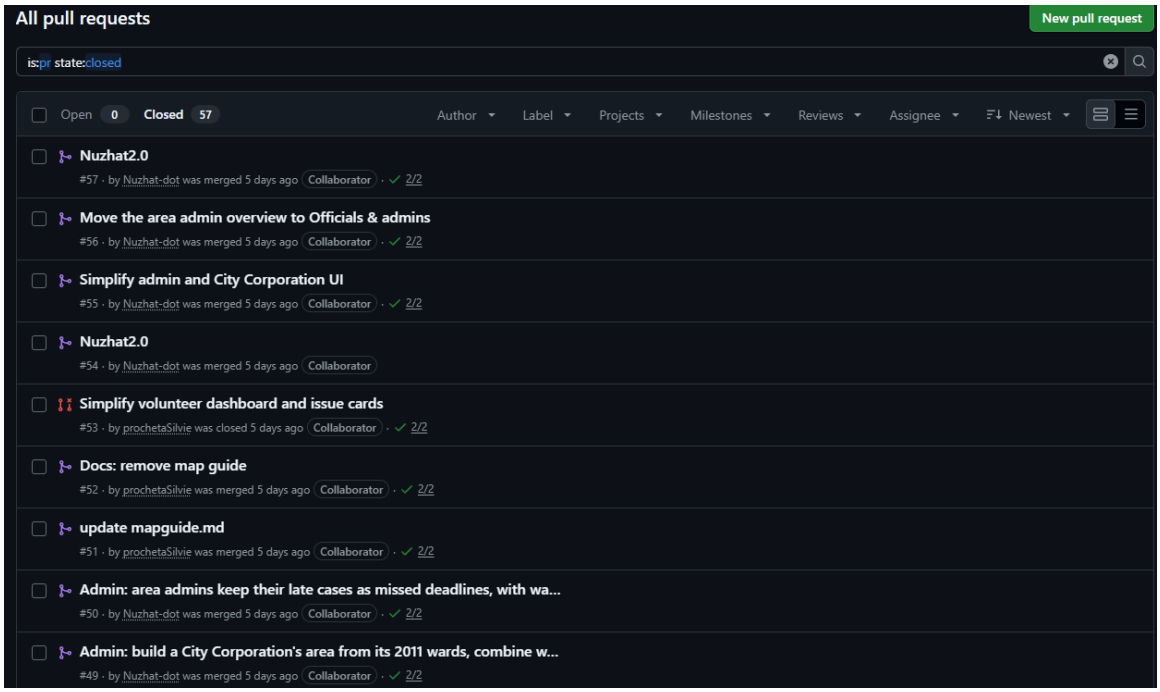
After the Implementation session, development continued in iterations. Table 9.2 lists them with the dates they were completed, taken from the dated migration file names and the commit history.

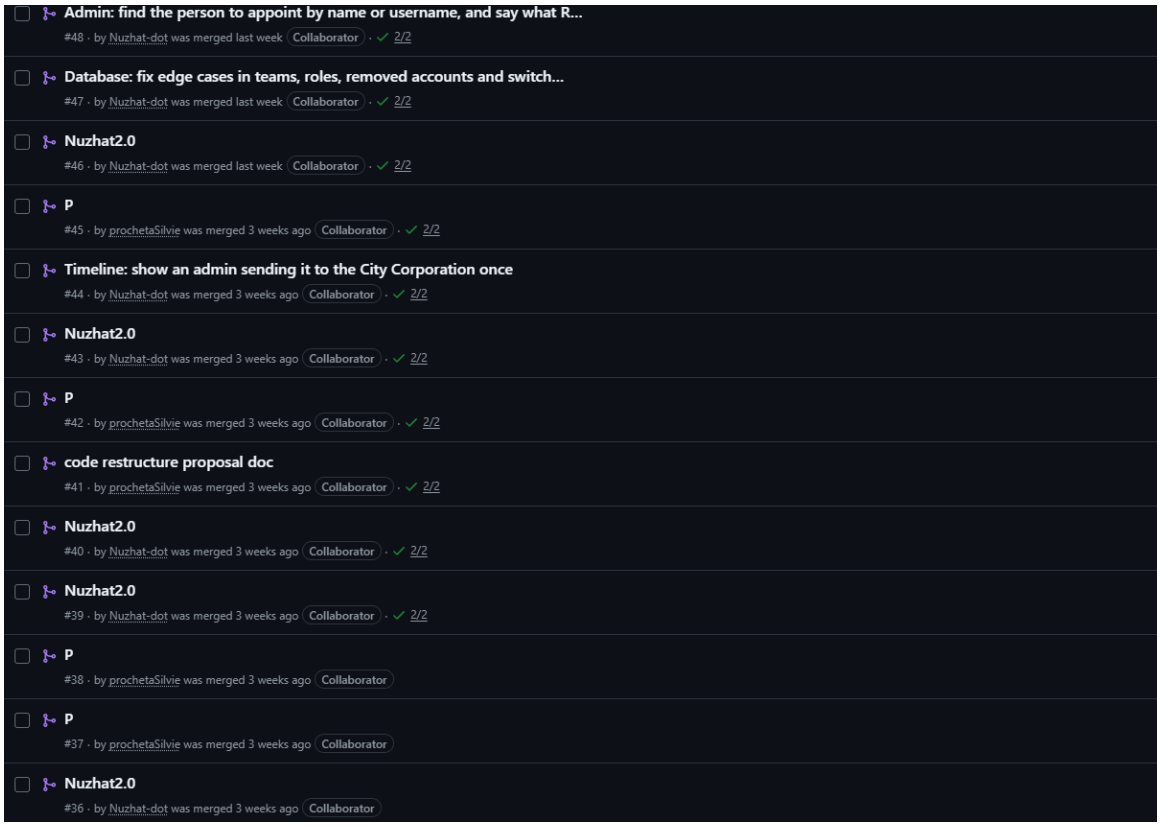
Development milestone	Completed
v1 core: schema, validation, volunteer workflow, map and heatmap (migrations 0001–0006)	3–5 Oct 2026
v2: roles, City Corporations, routing, review queue, teams, emergencies (0007–0011)	5–6 Oct 2026
Categories, map groups, emergencies vs issues, live evidence, spam and appeals (0012–0030)	7–13 Oct 2026
Web restructure into feature folders and service classes (`RESTRUCTURE_PLAN.md`)	11 Oct 2026
Roles and emergency fixes, privacy and uploads, area admins (0031–0052)	14–17 Oct 2026
Live evidence for issues, delete account, official sign-up, teams, edge cases (0053–0074)	18 Oct 2026
Final fixes, course repository layout, UI simplification (0075–0078)	25–28 Oct 2026

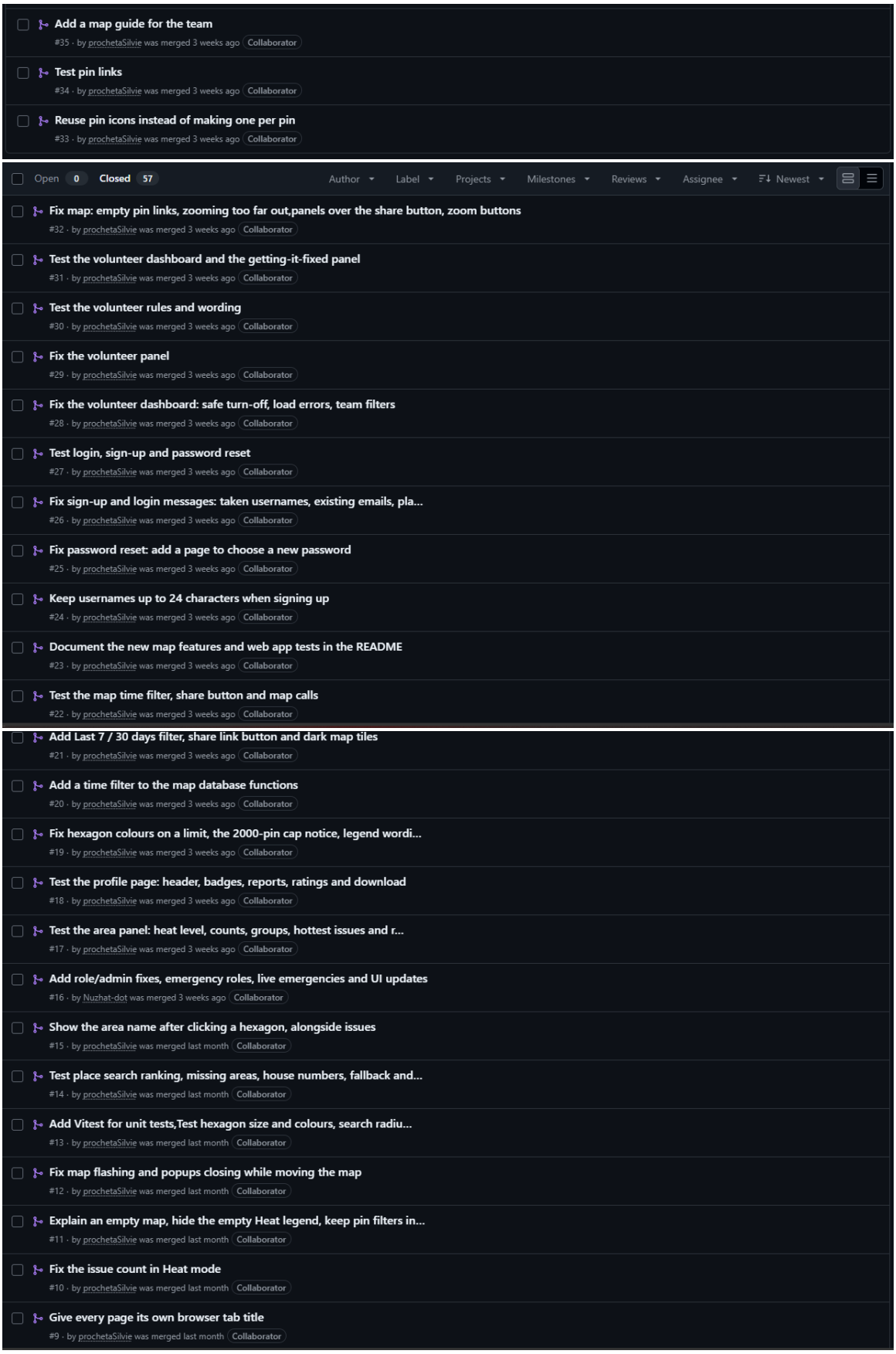
9.2. Task tracking and progress review

- Work was tracked through GitHub branches and **57 pull requests**; each pull request groups one piece of work and is reviewed when merged into `main`.
- The CI status (green or red) on every pull request shows whether the work is complete and safe to merge.
- Team guides (`doc/guides/`) record agreed plans, for example `RESTRUCTURE_PLAN.md` ("agreed by the team") and `END_TO_END_TEST.md` for review before the demonstration.

- We used GitHub itself as our task tracker: each task became a branch and a pull request, merged into `main` once CI passed. Figure 9.1 shows all 57 closed pull requests.






Add a map guide for the team
#35 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test pin links
#34 · by prochetaSilvie was merged 3 weeks ago Collaborator
Reuse pin icons instead of making one per pin
#33 · by prochetaSilvie was merged 3 weeks ago Collaborator
Open 0 Closed 57 Author Label Projects Milestones Reviews Assignee Newest
Fix map: empty pin links, zooming too far out,panels over the share button, zoom buttons
#32 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test the volunteer dashboard and the getting-it-fixed panel
#31 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test the volunteer rules and wording
#30 · by prochetaSilvie was merged 3 weeks ago Collaborator
Fix the volunteer panel
#29 · by prochetaSilvie was merged 3 weeks ago Collaborator
Fix the volunteer dashboard: safe turn-off, load errors, team filters
#28 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test login, sign-up and password reset
#27 · by prochetaSilvie was merged 3 weeks ago Collaborator
Fix sign-up and login messages: taken usernames, existing emails, pla...
#26 · by prochetaSilvie was merged 3 weeks ago Collaborator
Fix password reset: add a page to choose a new password
#25 · by prochetaSilvie was merged 3 weeks ago Collaborator
Keep usernames up to 24 characters when signing up
#24 · by prochetaSilvie was merged 3 weeks ago Collaborator
Document the new map features and web app tests in the README
#23 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test the map time filter, share button and map calls
#22 · by prochetaSilvie was merged 3 weeks ago Collaborator
Add Last 7 / 30 days filter, share link button and dark map tiles
#21 · by prochetaSilvie was merged 3 weeks ago Collaborator
Add a time filter to the map database functions
#20 · by prochetaSilvie was merged 3 weeks ago Collaborator
Fix hexagon colours on a limit, the 2000-pin cap notice, legend wordi...
#19 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test the profile page: header, badges, reports, ratings and download
#18 · by prochetaSilvie was merged 3 weeks ago Collaborator
Test the area panel: heat level, counts, groups, hottest issues and r...
#17 · by prochetaSilvie was merged 3 weeks ago Collaborator
Add role/admin fixes, emergency roles, live emergencies and UI updates
#16 · by Nuzhat-dot was merged 3 weeks ago Collaborator
Show the area name after clicking a hexagon, alongside issues
#15 · by prochetaSilvie was merged last month Collaborator
Test place search ranking, missing areas, house numbers, fallback and...
#14 · by prochetaSilvie was merged last month Collaborator
Add Vitest for unit tests,Test hexagon size and colours, search radiu...
#13 · by prochetaSilvie was merged last month Collaborator
Fix map flashing and popups closing while moving the map
#12 · by prochetaSilvie was merged last month Collaborator
Explain an empty map, hide the empty Heat legend, keep pin filters in...
#11 · by prochetaSilvie was merged last month Collaborator
Fix the issue count in Heat mode
#10 · by prochetaSilvie was merged last month Collaborator
Give every page its own browser tab title
#9 · by prochetaSilvie was merged last month Collaborator

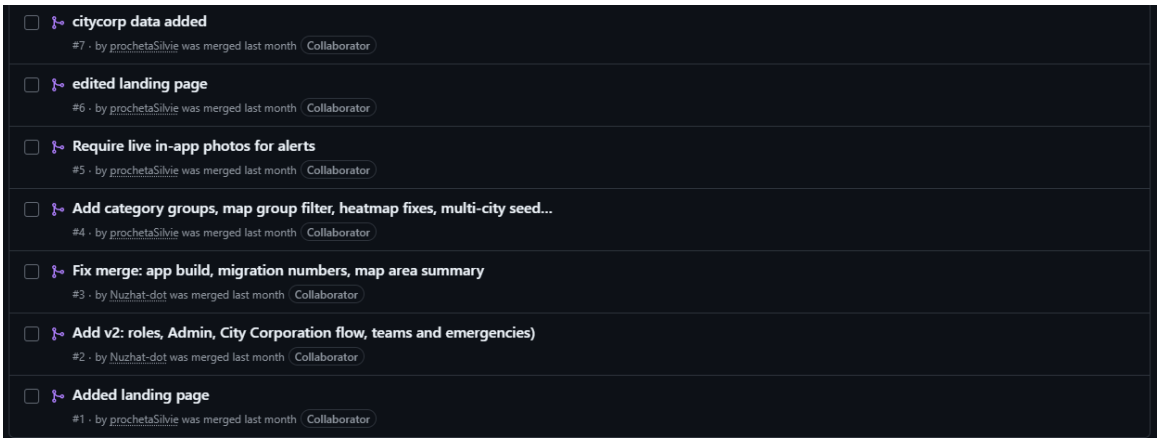


Figure 9.1: Closed pull requests on GitHub (#1 to #57), used as our task tracker

9.3. Scope and schedule changes and how they were handled

Change	Reason	How it was decided and communicated
PERN stack replaced by Supabase	Rules in the database are safer and less work (Section 5.6)	Discussed in a team meeting after the Design session; recorded in the README and the new migrations
Area admins added (one per City Corporation)	One admin cannot moderate a whole country	Migrations 0044–0052; documented in the README
Analytics dashboard narrowed to the public City Corporation record and the area admin overview	Time; these give the most useful numbers	Agreed in the team chat and noted in the README
Full Bangla UI, Google/phone login and the "Help your city" module postponed	Time; core report-to-fix loop came first	Listed under "Not built yet" in the README
Complaint reference removed (0059); "Leave as is" removed (27 Oct)	Simplify the officials' and admins' flow	Commit messages and pull requests

9.4. Effort, infrastructure and third-party cost estimates

Item	Estimate
Team effort	3 members from the SRS session (8 Jul 2026) to the final presentation (28 Oct 2026); 683 commits and about 37,600 lines of TypeScript, SQL and shell
Database, auth, storage, realtime (Supabase)	Free tier for development and a pilot; about USD 25 per month on the Pro plan when real users join (approximate; check supabase.com/pricing)
Web hosting (static build)	Free tier of a static host for a pilot
Map tiles and place search (OpenStreetMap, Photon, Nominatim)	Free under fair-use policies; a paid tile or geocoding provider would be needed at city scale

Item	Estimate
Domain name	About BDT 1,500 per year for a .com domain (approximate; optional for a pilot)
CI (GitHub Actions)	Free for this repository's usage

9.5. Feasibility

The solution was technically feasible within the timeline because (1) Supabase provided authentication, storage, real-time updates and scheduled jobs, so the team only had to write the rules and the interface; (2) the work split cleanly into three areas (reporting and validation, map and heatmap, roles and City Corporations); and (3) automated tests and CI let three people change the same database safely. The delivered system (Section 11) confirms this: 31 of the 36 SRS requirements were fully completed and the other 5 partly by the presentation date.

Self-assessment

Course outcome question	Our answer and evidence
CO7: Evidence of iterative planning, milestones, task tracking and progress review?	Tables 9.1 and 9.2 (sessions held on their planned dates, then development milestones from dated migrations); Section 9.2 (branches, 57 pull requests, CI status, team guides).
CO7: When schedule or scope changed, how did the team reprioritise and communicate?	Table 9.3 lists each change, the reason and how it was recorded.
CO7: Reasonable effort, infrastructure and third-party cost estimates?	Table 9.4.
CO7: Is the solution technically feasible within the timeline and resources?	Section 9.5, confirmed by the completed requirements in Section 11.

10. Teamwork and Individual Contributions

10.1. Responsibilities and evidence

The work was divided by feature, so each member owned a complete part of the system: its database tables and functions, its web pages and its tests. Migration numbers are the final numbers in `code/supabase/migrations/`.

Member	Responsibilities	Evidence in version control
Procheta Silvie (230042114)	• **Map and heatmap:** hexagon, heat and pin views; category and time filters; shareable map link; dark map tiles • **Area summary and place search:** hexagon and pin area summaries; OpenStreetMap place search (`PlaceSearchService`) • **Issue categories:** category groups and the map group filter; category corrections • **Emergency alerts:** emergencies vs issues; turning an issue into an alert; live photos for alerts • **Landing page** • **Web restructure** into feature folders and service classes • **Web tests:** set up Vitest and wrote most of the 521 web tests	346 commits on branch `p` About 40 pull requests (#1, #4–#35, #51–#53), e.g. #13 (Vitest), #21 (time filter), #41 (restructure) Key commit: `1e8c538` Migrations: 0006, 0012–0021, 0026–0027, 0034–0035
Tasnia Farzana (230042124)	• **Roles and permissions:** role tables, admin checks, reasons for every admin action • **Routing to City Corporations:** finding the responsible City Corporation, escalation with target dates • **Admin panel and review queue:** super admins and one area admin per City Corporation; passing late cases up • **City Corporation officials:** sign-up by email domain, approval, official dashboard, releasing tasks • **City Corporation areas:** drawing an area from thanas, wards or freehand • **Volunteer teams:** asking for volunteers, joining, check-in, handing over the lead • **Emergency roles:** official updates, ending alerts, removing fake alerts • **Tests:** SQL scenarios `scenario_v2.sql` and `scenario_v4.sql`	119 commits on branches `Nuzhat` and `Nuzhat2.0` 17 pull requests (#2, #3, #16, #36, #39, #40, #43, #44, #46–#50, #54–#57), e.g. #2 (v2 roles and City Corporation flow), #16, #57 Key commit: `4d7f2a7` Migrations: 0007–0010, 0031–0033, 0044–0052, 0057–0078
Aksan Anan Ria (230042154)	• **Core database:** tables, business rules as database functions, read views and security, storage, scheduled jobs • **Issue reporting and feed:** report form, duplicate check, the feed • **Community validation:** weighted votes, on-site confirmations, validation thresholds • **Spam control:** fake-report penalties, posting pause, appeals, daily flag limit • **Login and authentication:** sign-up, login, password reset • **Privacy and security:** untraceable uploads, closed functions, delete account, live photos for fixes • **CI and documentation:** GitHub Actions, README, repository layout; SQL scenarios `scenario.sql` and `scenario_v3.sql`	154 commits, mostly direct to `main` Key commits: `73a3155` (schema), `792ac38` (business logic) Migrations: 0001–0005, 0011, 0022–0025, 0028–0030, 0036–0043, 0053–0056

10.2. Coordinating interdependent modules

- **Shared contract:** the database functions and the TypeScript models in `models/` are the interface between members; a change to a function's parameters is made together with its service method.
- **One ordered database history:** each member added numbered migrations; when two branches added migrations at the same time, they were renumbered on merge so everyone builds the same database.
- **Written rules:** `doc/guides/ARCHITECTURE.md` says where new code goes, so features written by different members follow one structure.
- **CI as the integration gate:** a pull request is merged only when lint, 521 web tests, the build, all migrations and the SQL scenarios pass.
- **Example of integration:** the map (Procheta) shows issues routed by City Corporation areas (Tasnia) that were validated by the community rules (Aksan); all three meet in the `issues` table and its read views.

10.3. Disagreements, blockers and fair participation

- **Disagreement: which back end to use.** Part of the team wanted to keep the PERN stack we had planned, because it was familiar; others argued that rules inside the database would be safer and save time. We compared both on a small feature, agreed on Supabase, and recorded the reasons (Section 5.6).
- **Blocker: migration conflicts.** Because all three of us added database migrations in parallel, two branches sometimes used the same number. We solved this by renumbering the later migrations on merge and letting CI rebuild the whole database from scratch to prove the order still worked.
- **Blocker: one very large admin page and API file.** `AdminPage.tsx` (1,102 lines) and `lib/api.ts` (471 lines) made parallel work hard and caused merge conflicts; the feature-folder restructure removed this.
- The web restructure was written up as a plan and applied only after it was "agreed by the team" (`RESTRUCTURE_PLAN.md`).
- Each member owned a complete vertical slice (database + web + tests), so every member can demonstrate and modify their own part.

Self-assessment

Course outcome question	Our answer and evidence
CO5: Can every member explain assigned tasks and show how they are reflected in version control?	Table 10.1 lists what each member built, with their branches, pull requests, commits and migrations.

Course outcome question	Our answer and evidence
CO5: How were disagreements and blockers resolved, and was participation fair?	Section 10.3.
CO5: How did members coordinate interdependent modules and integration?	Section 10.2.

11. Results, Limitations, and Future Work

11.1. Implementation status of each requirement

Of the 36 functional requirements in our SRS, **31 are completed** and **5 are partially completed**; none were dropped. All 10 non-functional requirements are met except two that are partial.

ID	Requirement	Status	How it was implemented
FR-1	Phone Registration	Partial	Registration with email and password; phone number with OTP not built
FR-2	Login	Completed	Email and password; JWT session
FR-3	Profile Management	Completed	Name, photo, bio and area
FR-4	Password Recovery	Completed	Reset link by email
FR-5	Volunteer Application	Partial	Volunteer mode from the same account (account ≥ 72 h, home area required); skills and availability not collected
FR-6	Two Language Support	Partial	Category names in Bangla and English; the interface is in English
FR-7	Role Based Access	Completed	Visitor, citizen, volunteer, official, area admin, super admin; checked in every database function
FR-8	Report Submission with Attachments	Completed	1–5 photos or videos
FR-9	Categorised Issue Reporting	Completed	Category groups, subgroups and categories
FR-10	Duplicate Report Detection	Completed	50 m, same category, last 30 days
FR-11	Report Location Management	Completed	GPS or hand-placed pin; editable until validated
FR-12	Report Status Tracking	Completed	Status badge and timeline on every issue
FR-13	View Previous Reports	Completed	On the user's profile
FR-14	Confirm Existing Issue	Completed	"I see this too" with a live photo within 200 m
FR-15	Report Priority	Completed	Severity by community vote (median of 3)

ID	Requirement	Status	How it was implemented
FR-16	Volunteer Task Management	Completed	Dashboard, accept, 72-hour lock, release
FR-17	Task Progress Update	Completed	Progress posts restart the lock
FR-18	Map-Based Report Display	Completed	Hexagons, heat layer and clustered pins
FR-19	Emergency Report Marking	Completed	Emergency alerts; an issue can be turned into an alert
FR-20	Notifications	Completed	Live in-app notifications
FR-21	Emergency Flagging and Verification	Completed	Neighbour confirmations, live evidence, official or admin review
FR-22	Verify Report Evidence	Completed	On-site confirmations and live photos; fixes confirmed by residents
FR-23	Receive Nearby Issue Alerts	Completed	Volunteers within 5 km of their home area are notified
FR-24	Comment on a Report	Completed	Comments; every issue and map view has a shareable link
FR-25	Search and Filter Reports	Completed	Map search, category and time filters
FR-26	Public Analytics Dashboard	Partial	Public City Corporation record, leaderboard, area summary; no trend charts
FR-27	Escalate Unresolved Issue	Completed	Routing to City Corporations with target times and an Overdue label
FR-28	Impact Info on Reported Issue	Completed	Size, severity and description
FR-29	Add Notes	Completed	Notes on progress updates and releases
FR-30	Additional Evidence	Completed	Confirmation and progress photos
FR-31	Filter and Sort Reports	Completed	Category dropdown, newest first, time filter
FR-32	Save Important Reports	Completed	Follow a report to save it and get its updates
FR-33	Volunteer Profile Management	Partial	Home area and search radius; no skills or availability
FR-34	Volunteer Feedback and Rating	Completed	1–5 stars from the reporter; reputation and leaderboard
FR-35	Task Participation Requests	Completed	Join a team, "Ask for more volunteers"
FR-36	Deactivate Account	Completed	Delete my account; reports stay without a name

NFR	Status	Evidence
NFR-1 Upload validation	Completed	File count, type and video-length checks (`media/lib/mediaRules.ts`, tested)
NFR-2 Updates within seconds	Partial	Notifications are live; the feed and map refresh when reopened or refocused

NFR	Status	Evidence
NFR-3 Bangla and English	Partial	Category names only
NFR-4 Secure credentials	Completed	Passwords hashed by Supabase Auth; private columns hidden by views
NFR-5 Emergency verification	Completed	Signed-in reporters, live evidence, established confirmers
NFR-6 Fast duplicate detection	Completed	50 m search on a spatial index
NFR-7 Reliable persistence	Completed	PostgreSQL transactions; weekly backups (`BACKUPS.md`)
NFR-8 Responsive on mobile	Completed	Responsive layout; photo compression
NFR-9 Clear errors	Completed	Every database error carries a code and a readable message
NFR-10 Role-based access	Completed	Checked inside each database function; tested in `scenario_v2.sql` and `scenario_v4.sql`

Beyond the SRS, the final system also has City Corporation officials and area admins, volunteer teams, live photo evidence, fake-report penalties with appeals, a "Is this still there?" check, and a public record for each City Corporation.

11.2. Known limitations

- The app makes City Corporation delays visible but cannot force an authority to act.
- Live photo evidence needs a phone camera; for a laptop demo it is switched off with `live_issue_evidence = false`.
- Some React components are long (the largest has 397 lines); Section 6.4 explains why.
- Business logic in SQL is harder to debug than application code; the SQL scenario tests reduce this risk.
- Free OpenStreetMap and search services have fair-use limits that a city-wide launch would exceed.

11.3. Future work

- Phone registration with OTP, and a full Bangla interface (FR-1, FR-6).
- Volunteer skills and availability (FR-5, FR-33) and a trends dashboard (FR-26).
- Image-similarity duplicate detection and splitting the largest components.
- Integration with City Corporation complaint systems through an Open311-style API, and a mobile app with offline reporting.

Self-assessment

Course outcome question	Our answer and evidence
CO6: Would an informed reader understand the problem, requirements, design decisions, implementation status and limitations from the report alone?	The report follows the path problem (Section 2), requirements (4), design and alternatives (5), implementation (6), and status and limitations (11), with diagrams and a glossary; the README explains how to build, run and test the code.

12. Glossary and References

12.1. Glossary

Term	Meaning
API	Application Programming Interface: the way one program asks another for data or actions
CI/CD	Continuous Integration / Continuous Deployment: running tests (and deploying) automatically on every change
City Corporation	The local government body of a Bangladeshi city (e.g. DNCC, DSCC)
Escalated	An issue handed to the City Corporation responsible for its location
GIS	Geographic Information System: software for working with locations and maps
GiST index	A database index that makes spatial searches (e.g. "which area contains this point?") fast
Heatmap / hexagon map	A map that shows where problems are concentrated, as colour or as coloured hexagons
JWT	JSON Web Token: a signed token that proves who is logged in
Migration	A numbered SQL file that changes the database; all migrations run in order
PostGIS	An extension that adds geographic types and functions to PostgreSQL
RLS	Row-Level Security: database rules that decide which rows a user may read
RPC	Remote Procedure Call: calling a database function from the browser
SECURITY DEFINER	A database function that runs with its owner's rights after checking the caller
SPA	Single-Page Application: a website that updates without reloading the page
SRS	Software Requirements Specification
Supabase	A hosted service that provides PostgreSQL, login, file storage and real-time updates
Thana / ward	Administrative subdivisions of a Bangladeshi city
Validated	An issue with enough weighted community support to be treated as real

12.2. References

1. mySociety, FixMyStreet. https://www.fixmystreet.com
2. SeeClickFix. https://seeclickfix.com
3. Ushahidi. https://www.ushahidi.com
4. Open311 GeoReport v2 specification. https://www.open311.org
5. PostGIS Documentation. https://postgis.net/documentation/
6. Supabase Documentation (Auth, Row Level Security, Database Functions, Realtime, Storage). https://supabase.com/docs
7. PostgreSQL Documentation. https://www.postgresql.org/docs/
8. Leaflet Documentation. https://leafletjs.com
9. OpenStreetMap Copyright and Licence (ODbL). https://www.openstreetmap.org/copyright
10. Nominatim Usage Policy. https://operations.osmfoundation.org/policies/nominatim/
11. Uber Engineering, "H3: Uber's Hexagonal Hierarchical Spatial Index", 2018. https://www.uber.com/blog/h3/
12. D. H. Douglas and T. K. Peucker, "Algorithms for the reduction of the number of points required to represent a digitized line or its caricature", The Canadian Cartographer, 10(2), 1973.
13. geoBoundaries: Bangladesh administrative boundaries (CC BY 3.0 IGO). https://www.geoboundaries.org
14. United Nations, Sustainable Development Goals 3, 11 and 16. https://sdgs.un.org/goals
15. React Documentation. https://react.dev
16. Vitest Documentation. https://vitest.dev
17. AmarShohor source repository. https://github.com/aksanX/SPL_II_AmarShohor

Self-assessment

Course outcome question	Our answer and evidence
CO6: Have the documents and presentation considered the audience's background, language and learning needs?	The Executive Summary is written for non-technical readers, every acronym is in the Glossary, diagrams support each technical section, and the README gives step-by-step build, run and test instructions.